

Mitali Bhatia

D20C 04

Blockchain

Experiment No 01

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output:

Program #1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

*** Enter a string: mitali
Hash: 4fd4129e2d4d4423d3c2f522bb92c721493752cad5b82faada980b1bae1fd4b6

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

Enter a string: mitalibhatia
Enter the nonce: 2
Hash Code: 507f2c07b9ca47df6d221bcba2b5ec06816bf25527dbf9e021ce9f6ff67d72a8

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = input("Enter a String: ")
num_zeros = int(input("Enter the number of leading zeros: "))

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

```
Enter a String: mitalibhatia
Enter the number of leading zeros: 2
Hash: 00a464c9fe2c239676a3bf98501651454ac9cfee700ff48f3fb989e9b53bcf00
Input String: mitalibhatia
Leading Zeros: 2
Expected Nonce: 26
```

Program #4: Generating Merkle Tree for given set of Transactions

```
import hashlib

# Sample transactions
transactions = [
    "Mitali -> Jai : $180",
    "Vanshika -> Sarthak : $425",
    "Jai -> Prishita : $275",
    "Sarthak -> Vanshika : $550",
    "Prishita -> Mitali : $320"
]

# Hash a transaction
def hash_transaction(transaction):
    return hashlib.sha256(transaction.encode('utf-8')).hexdigest()

# Hash all transactions before combining them
hashed_transactions = []

print("Initial Transaction Hashes:")
for transaction in transactions:
    transaction_hash = hash_transaction(transaction)
    hashed_transactions.append(transaction_hash)

    print(transaction)
    print("Hash:", transaction_hash)
    print()

# Construct the Merkle Tree
current_level = hashed_transactions

level = 1

while len(current_level) > 1:
    print("----- Merkle Tree Level", level, "-----")

    next_level = []
```

```

for i in range(0, len(current_level), 2):

    # If there is an odd number of hashes,
    # duplicate the last hash
    left = current_level[i]

    if i + 1 < len(current_level):
        right = current_level[i + 1]
    else:
        right = left

    # Combine the two hashes
    combined = left + right

    # Hash the combined hashes
    parent_hash = hashlib.sha256(
        combined.encode('utf-8')
    ).hexdigest()

    next_level.append(parent_hash)

    print("Left Hash :", left)
    print("Right Hash:", right)
    print("Parent Hash:", parent_hash)
    print()

    current_level = next_level
    level += 1

# Merkle Root
merkle_root = current_level[0]

print("=====")
print("Merkle Root Hash:", merkle_root)

```

Initial Transaction Hashes:

Mitali -> Jai : \$180

Hash: 62ed84d7916aca7c312e6c5f51de2a694a4b71becdb7c575a4f065bd960c4858

Vanshika -> Sarthak : \$425

Hash: 70fd0d3aa4920fc874fa0ca5d59aaf2447e0516c2996568a685e2edeb5e99349

Jai -> Prishita : \$275

Hash: bb7014134fee7abc7d27c5f7b75533b761cf3ac5a24e6b6cf2b0ab454dab72a6

Sarthak -> Vanshika : \$550

Hash: 76692fdf2e855204448e03aa07494ea74f96395e6d8bbd71e12647482a9d76e4

Prishita -> Mitali : \$320

Hash: 9187b608fa42dc0610a125a6f63e9e31fb95bf7630dfaf4f08af74d5c58541b0

----- Merkle Tree Level 1 -----

Left Hash : 62ed84d7916aca7c312e6c5f51de2a694a4b71becdb7c575a4f065bd960c4858

Right Hash: 70fd0d3aa4920fc874fa0ca5d59aaf2447e0516c2996568a685e2edeb5e99349

Parent Hash: 6bf9b6b1a70a57e8f3dc6188c71b5b3156cb03b9ecc12aabe4df3b7bbf13fe5d

Left Hash : bb7014134fee7abc7d27c5f7b75533b761cf3ac5a24e6b6cf2b0ab454dab72a6

Right Hash: 76692fdf2e855204448e03aa07494ea74f96395e6d8bbd71e12647482a9d76e4

Parent Hash: 3b28d66e913aa0a84b4f7e2e06a5773db101c1dae75640cb32b40cbab95cc2a0

Left Hash : 9187b608fa42dc0610a125a6f63e9e31fb95bf7630dfaf4f08af74d5c58541b0

Right Hash: 9187b608fa42dc0610a125a6f63e9e31fb95bf7630dfaf4f08af74d5c58541b0

Parent Hash: bdc8b7284e6c7c854829c78ea34eaaaf1ba043616e2f8b7dd32b5af8c18ba6c13

----- Merkle Tree Level 2 -----

Left Hash : 6bf9b6b1a70a57e8f3dc6188c71b5b3156cb03b9ecc12aabe4df3b7bbf13fe5d

Right Hash: 3b28d66e913aa0a84b4f7e2e06a5773db101c1dae75640cb32b40cbab95cc2a0

Parent Hash: 9f66d06aea6065b65ca6e6ea50cf7a06be4c0a165108460acf129b022035cf20

Left Hash : bdc8b7284e6c7c854829c78ea34eaaaf1ba043616e2f8b7dd32b5af8c18ba6c13

Right Hash: bdc8b7284e6c7c854829c78ea34eaaaf1ba043616e2f8b7dd32b5af8c18ba6c13

Parent Hash: fa3de17bb1ce05a1ceb10aef3a3586443fb4feb5b6f1f4a8105a3e4e247e926a

----- Merkle Tree Level 3 -----

Left Hash : 9f66d06aea6065b65ca6e6ea50cf7a06be4c0a165108460acf129b022035cf20

Right Hash: fa3de17bb1ce05a1ceb10aef3a3586443fb4feb5b6f1f4a8105a3e4e247e926a

Parent Hash: 144b579bc81268f7acd83d37ce0c2d98b2fb4977dcdf20336d671be66b50c4bb

=====

Merkle Root Hash: 144b579bc81268f7acd83d37ce0c2d98b2fb4977dcdf20336d671be66b50c4bb

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.